SLAM : Projet Agora Sprint 5, demande 3

Mise en place d'une relation many to one avec le composant Doctrine

Auteur : *Yannick Narbey d'après un tutoriel de Marianne Delio*

Pré-requis :

- Symfony : contrôleurs, routes, templates twig
- Agora Sprint 5 demande 2 : création de BDD et génération de formulaire CRUD avec Doctrine

TABLE DES MATIERES

1. Contexte et objectifs ____ *2*

2. Réalisation ____ *3*

2.1 Création des entités tournoi et catégorie de tournois ____ **3**

2.1.1 Créer l'entité Tournoi ____ 3

2.1.2 Création de l'entité catégorie de tournois ____ 6

2.2 Création de la relation entre les entités tournoi et catégorie de tournois ____ **7**

2.2.1 Ajout d'une relation many to one ____ 8

2.2.2 Contrôler le code généré ; ____ 8

2.3 Création des tables Tournois et Catégorie par migration ____ **12**

2.3.1 Génèrer les migrations ____ 12

2.3.2 Octroi des droits en BDD ____ 12

2.3.3 Appliquer la migration : ____ 13

2.4 Génération des CRUD ____ **14**

2.4.1 Génération des CRUD de gestion des tournois ____ 14

2.4.2 Génération des CRUD de gestion des catégories de tournois ____ 15

2.4.3 Tests ____ 15

2.4.4 Convection du type de l'objet :CatTournois ____ 17

2.4.5 Ajout des nouveaux formulaires au menu ____ 19

2.4.6 Charter et franciser les nouveaux formulaires ____ 21

3. A vous de jouer ____ *25*

1. Contexte et objectifs

Le directeur de l'association Agora demande à pouvoir gérer des tournois qui sont regroupés par catégories.

L'ESN Logma est toujours en charge de mener les évolutions sur l'application Agora.

Dans cette itération, vous êtes chargé de mettre en œuvre ces nouvelles fonctionnalité à l'aide du composant Doctrine.

Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)
Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.
SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.

https://symfony.com/

Doctrine

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.

https://www.doctrine-project.org

Entité

Les **données** de la base de données sont des **objets**. Un objet dont l'enregistrement est confié à l'ORM se nomme une **entité** (entity en anglais). On dit également **persister une entité**, plutôt qu'enregistrer une entité.

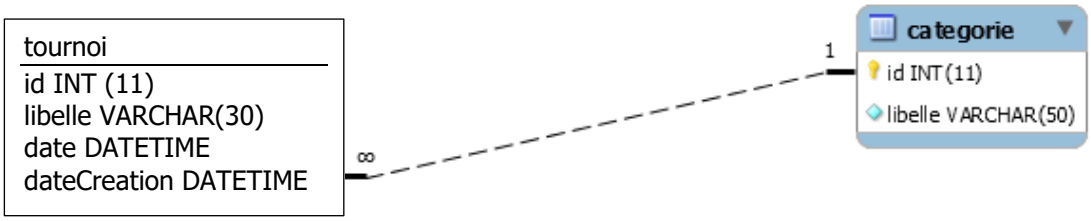
Attributs PHP et Doctrine

Doctrine utilise les attributs PHP 8 afin de définir la table correspondant à une entité, définir un identifiant (id) en auto-incrément, nommer les colonnes, etc. Ces informations se nomment les metadata de l'entité.
Ajouter les metadata à un objet, s'appelle **mapper l'objet, c'est**-à-dire faire le lien entre l'objet et la représentation physique qu'utilise Doctrine (une table de la base de données relationnelle).

Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

2. Réalisation

Un tournoi appartient à une seule catégorie et une catégorie comprend plusieurs tournois.



Pour Doctrine, ce cas correspond au **type d'association** : **ManyToOne / OneToMany**
Ce type d'association est courant et mappé avec une clé étrangère. Elle est vue des 2 côtés :

ManyToOne : vue depuis l'entité tournoi
OneToMany : vue depuis l'entité catégorie de tournois

Conception de la méthodologie :

- Créer l'entité Tournoi
- Créer l'entité Catégorie de tournois
- Déclarer la relation many to one entre tournoi et catégorie de tournois
- Lancer les migrations pour Tournoi et Categorie de tournois
- Générer les formulaires CRUD pour Tournoi
- Générer les formulaires CRUD pour Categorie de tournois
- Ajuster la présentation des formulaires

Pour éviter tout problème inutilement chronophage, respectez bien les noms indiqués dans le tutoriel, casse y comprise !

Comme l'application Agora actuelle (sprint 5) sera complétée par la suite, il est conseillé d'en faire une copie et de travailler sur la copie.

2.1 Création des entités tournoi et catégorie de tournois

2.1.1 Créer l'entité Tournoi

1. Les entités, autrement dit les classes métiers, peuvent être créées avec la commande **make:entity**. Il suffit de répondre aux différentes questions posées comme ci-dessous :

```
C:\wamp64\www\agora61>php bin/console make:entity

Class name of the entity to create or update (e.g. GentlePizza):
> Tournoi

created: src/Entity/Tournoi.php
created: src/Repository/TournoiRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.

New property name (press <return> to stop adding fields):
> libelle

Field type (enter ? to see all types) [string]:
> string

Field length [255]:
```

```
> 40

Can this field be null in the database (nullable) (yes/no) [no]:
> no

updated: src/Entity/Tournoi.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> date

Field type (enter ? to see all types) [string]:
> datetime

 Can this field be null in the database (nullable) (yes/no) [no]:
> no

updated: src/Entity/Tournoi.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> dateCreation

Field type (enter ? to see all types) [string]:
> datetime

Can this field be null in the database (nullable) (yes/no) [no]:
> no

updated: src/Entity/Tournoi.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>


 Success!


Next: When you're ready, create a migration with make:migration


C:\wamp64\www\agora61>
```

2. Un fichier src/Entity/Tournoi.php a été créé. Grâce aux annotations, chaque propriété est mappée (mise en correspondance) avec une colonne de la table de la base de données. Exemple pour la propriété libelle : `#[ORM\Column(length: 40)]` Les accesseurs (getters et setters) ont également été générés. Le code généré peut être modifié (ou développé) sans passer par la commande make qui n'est qu'une facilité.

Fichier agoram53/src/Entity/Tournoi.php

```php
<?php

namespace App\Entity;

use App\Repository\TournoiRepository;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: TournoiRepository::class)]
class Tournoi
{
```

```php
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 40)]
    private ?string $libelle = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $date = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $dateCreation = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getLibelle(): ?string
    {
        return $this->libelle;
    }

    public function setLibelle(string $libelle): static
    {
        $this->libelle = $libelle;

        return $this;
    }

    public function getDate(): ?\DateTimeInterface
    {
        return $this->date;
    }

    public function setDate(\DateTimeInterface $date): static
    {
        $this->date = $date;

        return $this;
    }

    public function getDateCreation(): ?\DateTimeInterface
    {
        return $this->dateCreation;
    }

    public function setDateCreation(\DateTimeInterface $dateCreation): static
    {
        $this->dateCreation = $dateCreation;

        return $this;
```

```
    }

    public function __construct()
    {
        $this->dateCreation = new \DateTime('now');
    }
}
```

Remarques :

`public function getId(): ?int`
PHP 7 ajoute le support des déclarations du type de retour. Les déclarations du type de retour spécifient le type de la valeur qui sera retournée par une fonction. Les mêmes types sont disponibles pour le typage du retour que pour le typage des arguments. Depuis PHP 7.1.0, les valeurs de retour peuvent être marquées comme nullable en préfixant le nom du type avec un point d'interrogation (?). **Ceci signifie que la fonction retourne soit le type spécifié soit NULL.**

`public function setLibelle(string $libelle): static`
Le nouveau type de retour statique (PHP 8) permet de réduire le type de retour à la classe appelée, ici Tournoi.

`\DateTimeInterface`
Interface pour les dates.
Une interface correspond à une liste d'opérations assurant un service cohérent et constituant un contrat à respecter par les classes réalisant l'interface. Une interface s'apparente à une classe abstraite

2.1.2 Création de l'entité catégorie de tournois

1. Passer la commande make:entity

```
C:\wamp64\www\agora53>php bin/console make:entity CatTournois
```

```
C:\Windows\System32\cmd.e

Microsoft Windows [version 10.0.26100.6899]
(c) Microsoft Corporation. Tous droits réservés.

C:\wamp64\www\agoram53>php bin/console make:entity CatTournois

 created: src/Entity/CatTournois.php
 created: src/Repository/CatTournoisRepository.php

 Entity generated! Now let's add some fields!
 You can always add more fields later manually or by re-running this command.

 New property name (press <return> to stop adding fields):
 > libelle

 Field type (enter ? to see all types) [string]:
 >

 Field length [255]:
 > 50

 Can this field be null in the database (nullable) (yes/no) [no]:
 >

 updated: src/Entity/CatTournois.php

 Add another property? Enter the property name (or press <return> to stop adding fields):
 >


 Success!

 Next: When you're ready, create a migration with php bin/console make:migration
```

2. Voici le code de la classe CatTournois qui a été généré :

Fichier agora53/src/Entity/CatTournois.php

```php
<?php

namespace App\Entity;

use App\Repository\CatTournoisRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: CatTournoisRepository::class)]
class CatTournois
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 50)]
    private ?string $libelle = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getLibelle(): ?string
    {
        return $this->libelle;
    }

    public function setLibelle(string $libelle): static
    {
        $this->libelle = $libelle;

        return $this;
    }
}
```

2.2 Création de la relation entre les entités tournoi et catégorie de tournois

Pour rappel, un tournoi appartient à une seule catégorie et une catégorie comprend plusieurs tournois.

Pour Doctrine, ce cas correspond au **type d'association** : **ManyToOne / OneToMany**
Ce type d'association est courant et mappé avec une clé étrangère. Elle est vue des 2 côtés :

ManyToOne : vue depuis l'entité tournoi
OneToMany : vue depuis l'entité catégorie de tournois

2.2.1 Ajout d'une relation many to one

Ajoutons une relation many to one avec CatTournois dans Tournoi, en utilisant la commande make :entity pour la mise à jour :

```
C:\wamp64\www\agora61>php bin/console make:entity
```

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram53>php bin/console make:entity

 Class name of the entity to create or update (e.g. VictoriousPuppy):
 > Tournoi

 Your entity already exists! So let's add some new fields!

 New property name (press <return> to stop adding fields):
 > categorie

 Field type (enter ? to see all types) [string]:
 > relation

 What class should this entity be related to?:
 > CatTournois

What type of relationship is this?
 ------------ ---------------------------------------------------------------------------
  Type         Description
 ------------ ---------------------------------------------------------------------------
  ManyToOne    Each Tournoi relates to (has) one CatTournois.
               Each CatTournois can relate to (can have) many Tournoi objects.

  OneToMany    Each Tournoi can relate to (can have) many CatTournois objects.
               Each CatTournois relates to (has) one Tournoi.

  ManyToMany   Each Tournoi can relate to (can have) many CatTournois objects.
               Each CatTournois can also relate to (can also have) many Tournoi objects.

  OneToOne     Each Tournoi relates to (has) exactly one CatTournois.
               Each CatTournois also relates to (has) exactly one Tournoi.
 ------------ ---------------------------------------------------------------------------

 Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:
 > ManyToOne

 Is the Tournoi.categorie property allowed to be null (nullable)? (yes/no) [yes]:
 >

 Do you want to add a new property to CatTournois so that you can access/update Tournoi objects from it - e.g. $catTourn
ois->getTournois()? (yes/no) [yes]:
 >

 A new property will also be added to the CatTournois class so that you can access the related Tournoi objects from it.

 New field name inside CatTournois [tournois]:
 >

 updated: src/Entity/Tournoi.php
 updated: src/Entity/CatTournois.php
```

```
 Add another property? Enter the property name (or press <return> to stop adding fields):
 >

 Success!

 Next: When you're ready, create a migration with php bin/console make:migration
```

2.2.2 Contrôler le code généré

Dans le fichier entity/Tournoi.php :

- C'est d'une annotation Doctrine #[ORM\ManyToOne(inversedBy: 'tournois')] qui indique que chaque objet Tournoi est lié à une seule catégorie (CatTournois) mais q'une catégorie peut avoir plusieurs tournois (c'est la partie inversedBy: 'tournois').

- un attribut catTournois a été ajouté avec son getter et son setter qui ne désigne pas un id mais un objet de la classe Categorie.

Fichier agora53/src/Entity/Tournoi.php

```php
<?php

namespace App\Entity;

use App\Repository\TournoiRepository;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: TournoiRepository::class)]
class Tournoi
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 40)]
    private ?string $libelle = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $date = null;

    #[ORM\Column(type: Types::DATETIME_MUTABLE)]
    private ?\DateTimeInterface $dateCreation = null;

    #[ORM\ManyToOne(inversedBy: 'tournois')]
    private ?CatTournois $categorie = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getLibelle(): ?string
    {
        return $this->libelle;
    }

    public function setLibelle(string $libelle): static
    {
        $this->libelle = $libelle;

        return $this;
    }

    public function getDate(): ?\DateTimeInterface
    //public function getDate(): ?DateTime
    {
        return $this->date;
```

```
    }

    //public function setDate(\DateTimeInterface $date): static
    public function setDate(\DateTimeInterface $date): static
    {
        $this->date = $date;

        return $this;
    }

    public function getDateCreation(): ?\DateTimeInterface
    {
        return $this->dateCreation;
    }

    public function setDateCreation(\DateTimeInterface $dateCreation): static
    {
        $this->dateCreation = $dateCreation;

        return $this;
    }

    public function __construct()
    {
        $this->dateCreation = new \DateTime('now');
    }

    public function getCategorie(): ?CatTournois
    {
        return $this->categorie;
    }

    public function setCategorie(?CatTournois $categorie): static
    {
        $this->categorie = $categorie;

        return $this;
    }
}
```

Dans le fichier entity/CatTournois.php :

- création d'une annotation Doctrine #[ORM\OneToMany(targetEntity: Tournoi::class, mappedBy: 'categorie')] qui indique qu'une catégorie peut contenir plusieurs tournois.
- 3 méthodes ont été ajoutées `getTournois(), addTournoi(), removeT().`

Fichier agora53/src/Entity/CatTournois.php

```
<?php

namespace App\Entity;

use App\Repository\CatTournoisRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;
```

```php
#[ORM\Entity(repositoryClass: CatTournoisRepository::class)]
class CatTournois
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 50)]
    private ?string $libelle = null;

    /**
     * @var Collection<int, Tournoi>
     */
    #[ORM\OneToMany(targetEntity: Tournoi::class, mappedBy: 'categorie')]
    private Collection $tournois;

    public function __construct()
    {
        $this->tournois = new ArrayCollection();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getLibelle(): ?string
    {
        return $this->libelle;
    }

    public function setLibelle(string $libelle): static
    {
        $this->libelle = $libelle;

        return $this;
    }

    /**
     * @return Collection<int, Tournoi>
     */
    public function getTournois(): Collection
    {
        return $this->tournois;
    }

    public function addTournoi(Tournoi $tournoi): static
    {
        if (!$this->tournois->contains($tournoi)) {
            $this->tournois->add($tournoi);
            $tournoi->setCategorie($this);
```

```
        }

        return $this;
    }

    public function removeTournoi(Tournoi $tournoi): static
    {
        if ($this->tournois->removeElement($tournoi)) {
            // set the owning side to null (unless already changed)
            if ($tournoi->getCategorie() === $this) {
                $tournoi->setCategorie(null);
            }
        }

        return $this;
    }
}
```

Explications :

```
$a === $b;   //  est vrai si : $a est égal à $b et qu'ils sont de même type
```

2.3 Création des tables Tournois et Catégorie par migration

2.3.1 Générer les migrations

Générer les migrations avec la commande make:migration :

```
C:\wamp64\www\agora53>php bin/console make:migration
```

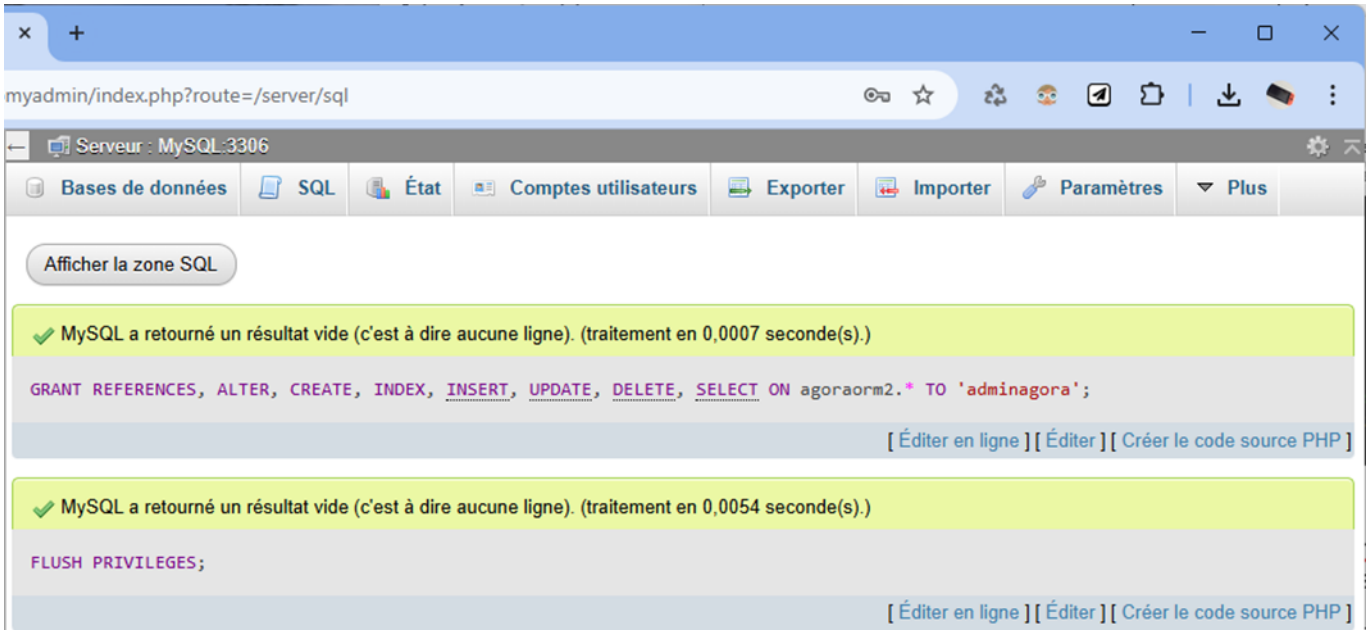
```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram53>php bin/console make:migration

 created: migrations/Version20251101170359.php

 Success!

 Review the new migration then run it with php bin/console doctrine:migrations:migrate
 See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html
```

2.3.2 Octroi des droits en BDD

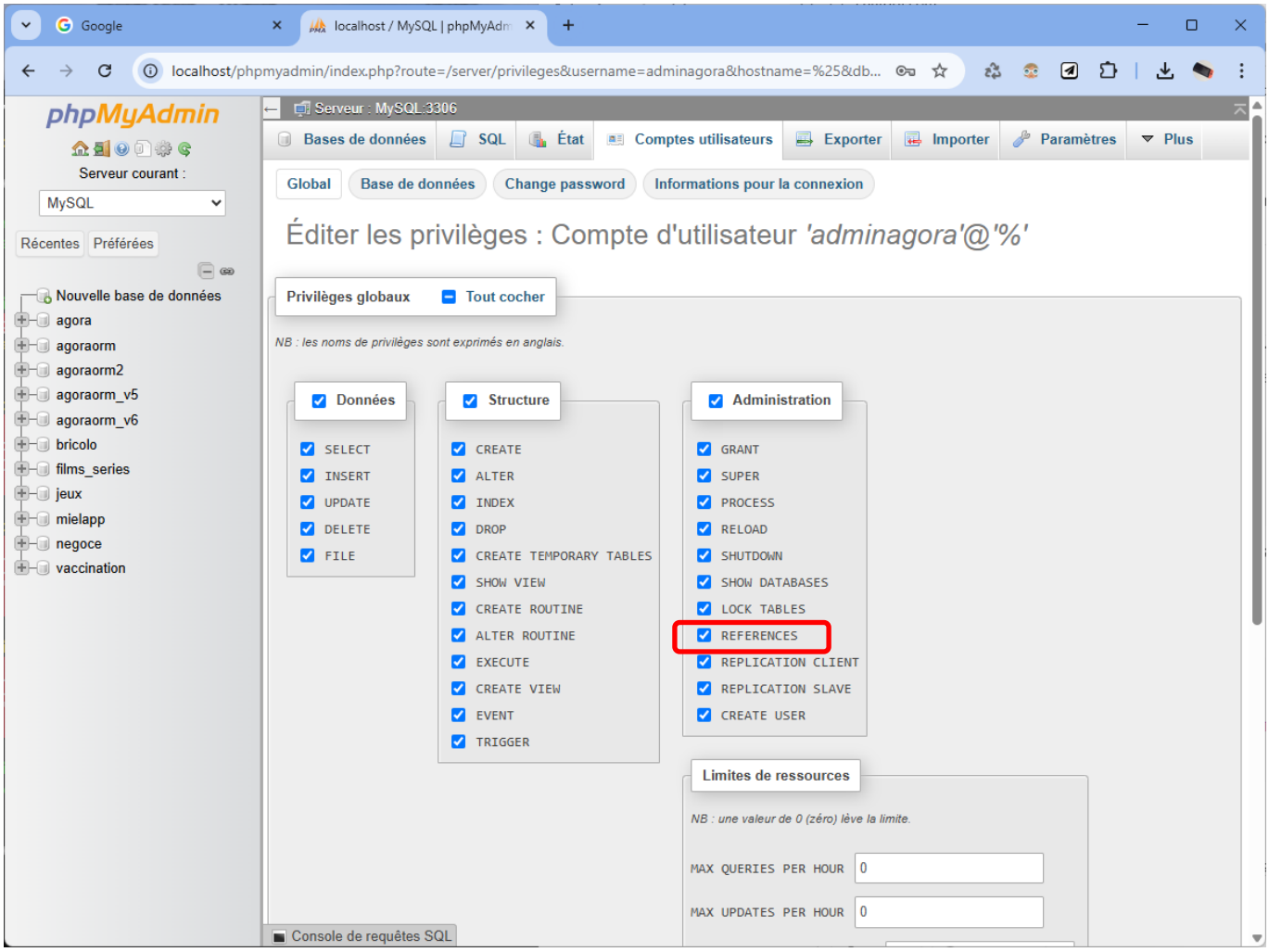
Pour que l'execution soit possible, il faut que l'utilisateur adminagora ait le droit de mettre en place des clés étrangères,

Passer la commande SQL suivante dans PHPMyAdmin pour vous en assurer :

```
GRANT REFERENCES, ALTER, CREATE, INDEX, INSERT, UPDATE, DELETE, SELECT ON agoraorm.* TO 'adminagora';
FLUSH PRIVILEGES;
```



Ou ajouter le droit « references » au travers de l'interface d'administration pour l'utilisateur adminagora.

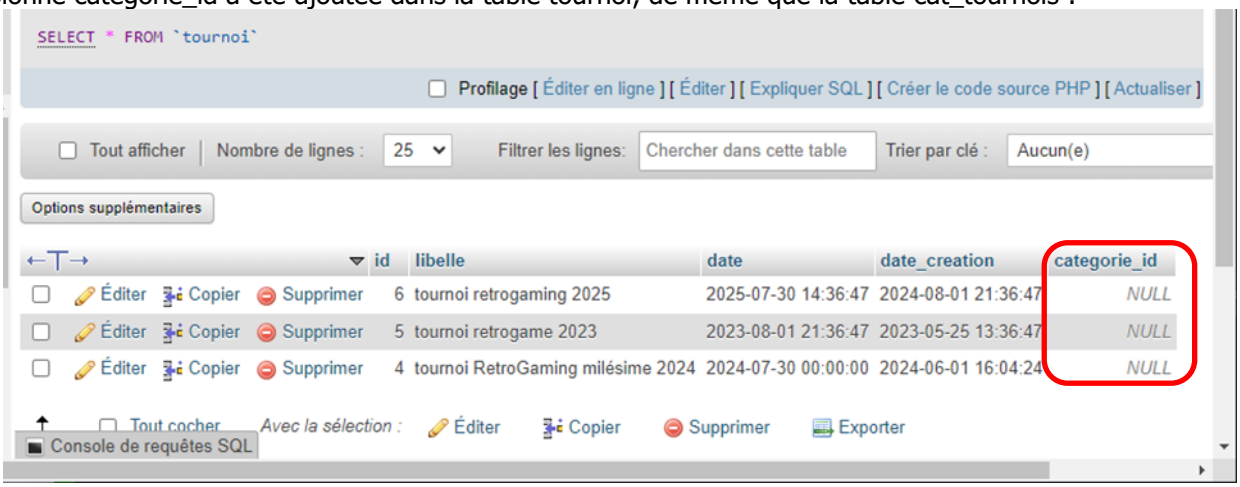


2.3.3 Appliquer la migration

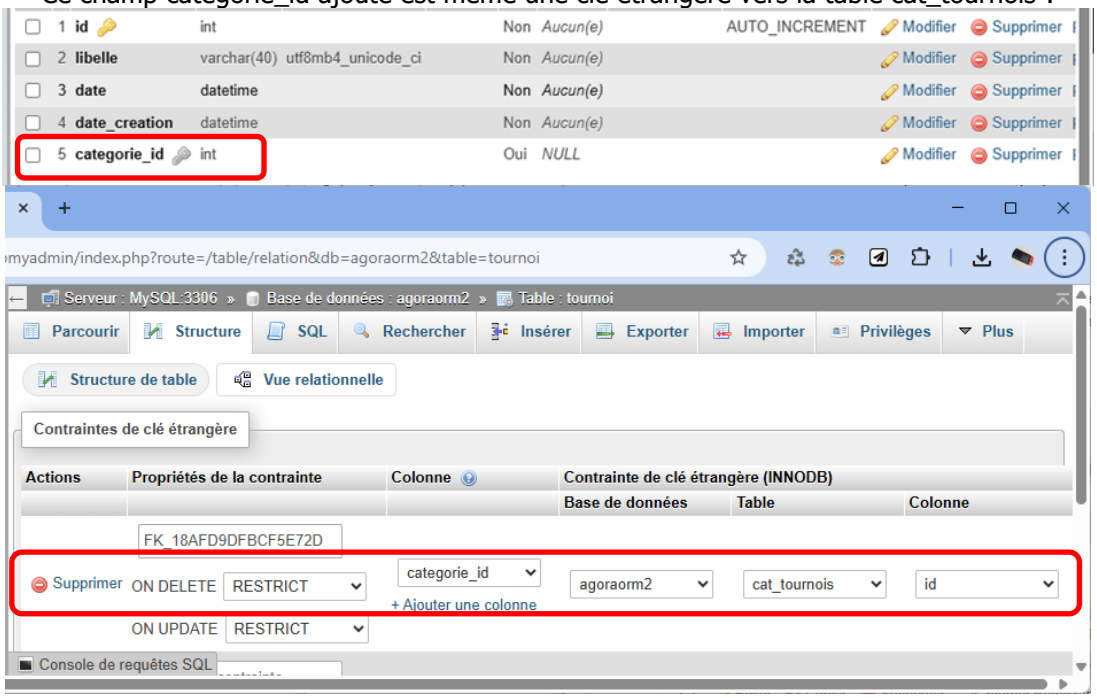
```
C:\wamp64\www\agora53>php bin/console doctrine:migrations:migrate
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram53>php bin/console doctrine:migrations:migrate

 WARNING! You are about to execute a migration in database "agoraorm2" that could result in schema changes and data loss
. Are you sure you wish to continue? (yes/no) [yes]:
 >

[notice] Migrating up to DoctrineMigrations\Version20251101170359
[notice] finished in 222ms, used 20M memory, 1 migrations executed, 4 sql queries

C:\wamp64\www\agoram53>
```

La colonne categorie_id a été ajoutée dans la table tournoi, de même que la table cat_tournois :



Ce champ categorie_id ajouté est même une clé étrangère vers la table cat_tournois :



2.4 Génération des CRUD

2.4.1 Génération des CRUD de gestion des tournois

Utiliser la commande make:crud

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram53>php bin/console make:crud Tournoi

 Choose a name for your controller class (e.g. TournoiController) [TournoiController]:
 >

 Do you want to generate tests for the controller?. [Experimental] (yes/no) [no]:
 >

 created: src/Controller/TournoiController.php
 created: src/Form/TournoiType.php
 created: templates/tournoi/_delete_form.html.twig
 created: templates/tournoi/_form.html.twig
 created: templates/tournoi/edit.html.twig
 created: templates/tournoi/index.html.twig
 created: templates/tournoi/new.html.twig
 created: templates/tournoi/show.html.twig

 Success!

 Next: Check your new CRUD by going to /tournoi/
```

Vérifier la génération des fichiers mentionnés ci-dessus.

2.4.2 Génération des CRUD de gestion des catégories de tournois

Utiliser la commande make :crud :

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram53>php bin/console make:crud CatTournois

 Choose a name for your controller class (e.g. CatTournoisController) [CatTournoisController]:
 >

 Do you want to generate tests for the controller?. [Experimental] (yes/no) [no]:
 > no

 created: src/Controller/CatTournoisController.php
 created: src/Form/CatTournoisType.php
 created: templates/cat_tournois/_delete_form.html.twig
 created: templates/cat_tournois/_form.html.twig
 created: templates/cat_tournois/edit.html.twig
 created: templates/cat_tournois/index.html.twig
 created: templates/cat_tournois/new.html.twig
 created: templates/cat_tournois/show.html.twig

 Success!

 Next: Check your new CRUD by going to /cat/tournois/
```

Vérifier la génération des fichiers mentionnés ci-dessus.

2.4.3 Tests

Vider le cache, pour forcer la prise en compte des modifications (nouvelles routes, nouveaux contrôleurs …)

.

```
C:\wamp64\www\agoram53>php bin/console cache:clear
```

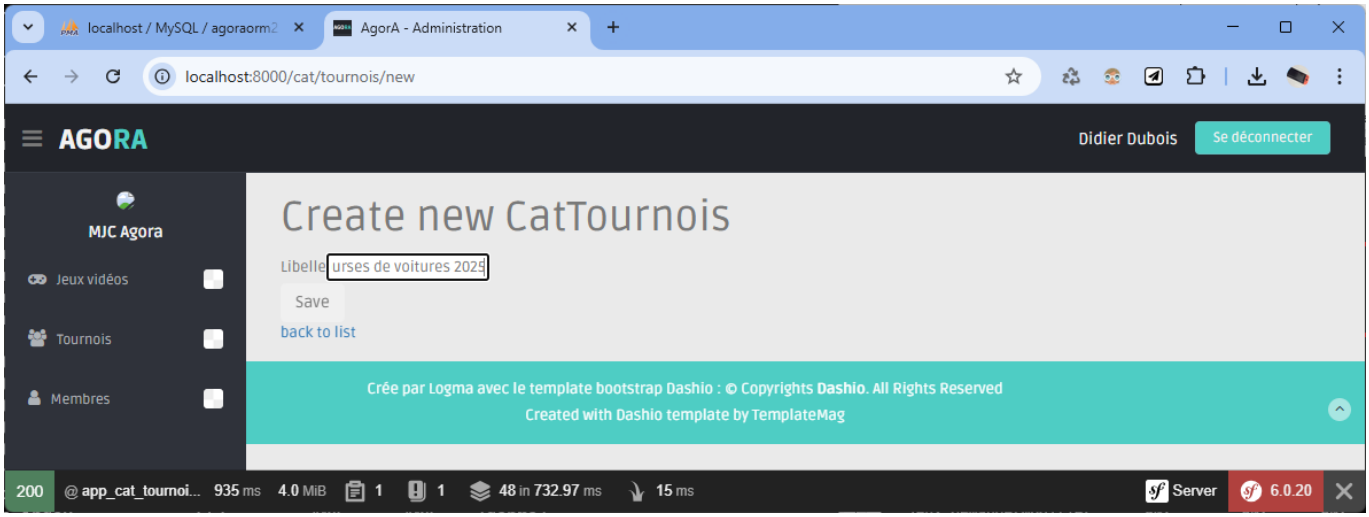
```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram53>php bin/console cache:clear

 // Clearing the cache for the dev environment with debug true

 [OK] Cache for the "dev" environment (debug=true) was successfully cleared.

C:\wamp64\www\agoram53>
```

Demander la liste des routes et retrouver les routes des nouveaux formulaires app_cat_tournois et app_tournoi :

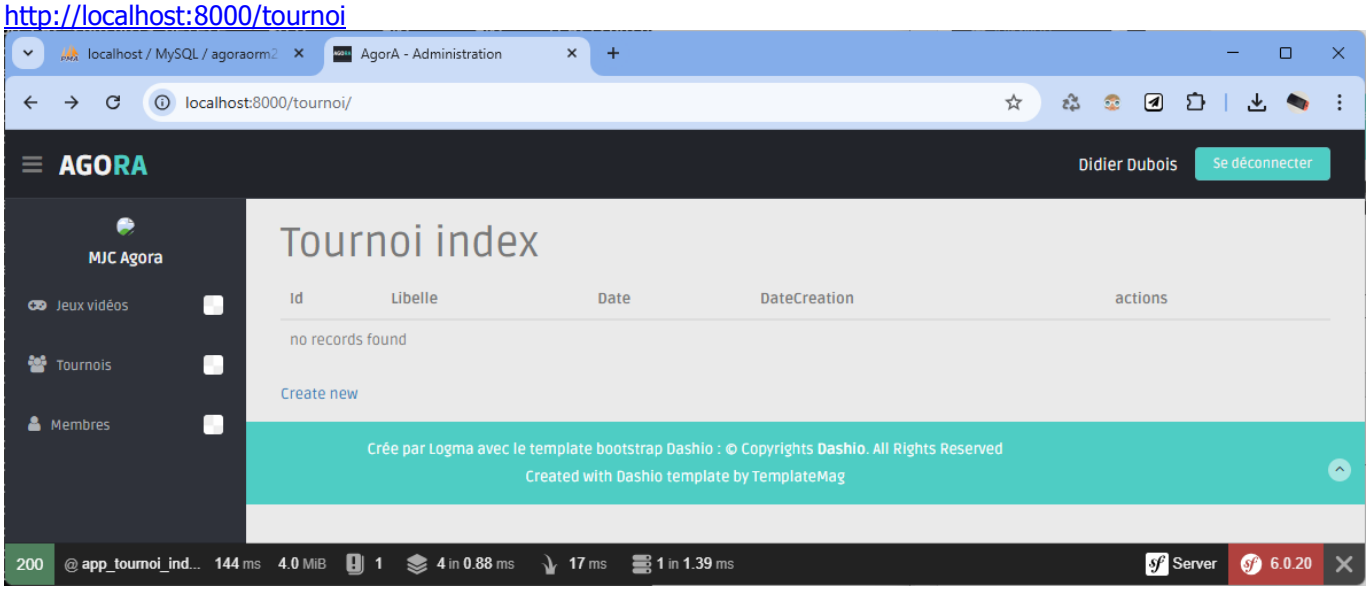
```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram53>php bin/console debug:router
 ------------------------- ---------- -------- ------ ----------------------------------
  Name                      Method     Scheme   Host   Path
 ------------------------- ---------- -------- ------ ----------------------------------
  _preview_error            ANY        ANY      ANY    /_error/{code}.{_format}
  _wdt                      ANY        ANY      ANY    /_wdt/{token}
  _profiler_home            ANY        ANY      ANY    /_profiler/
  _profiler_search          ANY        ANY      ANY    /_profiler/search
  _profiler_search_bar      ANY        ANY      ANY    /_profiler/search_bar
  _profiler_phpinfo         ANY        ANY      ANY    /_profiler/phpinfo
  _profiler_search_results  ANY        ANY      ANY    /_profiler/{token}/search/results
  _profiler_open_file       ANY        ANY      ANY    /_profiler/open
  _profiler                 ANY        ANY      ANY    /_profiler/{token}
  _profiler_router          ANY        ANY      ANY    /_profiler/{token}/router
  _profiler_exception       ANY        ANY      ANY    /_profiler/{token}/exception
  _profiler_exception_css   ANY        ANY      ANY    /_profiler/{token}/exception.css
  accueil                   ANY        ANY      ANY    /
  app_cat_tournois_index    GET        ANY      ANY    /cat/tournois/
  app_cat_tournois_new      GET|POST   ANY      ANY    /cat/tournois/new
  app_cat_tournois_show     GET        ANY      ANY    /cat/tournois/{id}
  app_cat_tournois_edit     GET|POST   ANY      ANY    /cat/tournois/{id}/edit
  app_cat_tournois_delete   POST       ANY      ANY    /cat/tournois/{id}
  connexion_valider         ANY        ANY      ANY    /connexion/valider
  deconnexion               ANY        ANY      ANY    /deconnexion
  app_genre_index           GET        ANY      ANY    /genre/
  app_genre_new             GET|POST   ANY      ANY    /genre/new
  app_genre_show            GET        ANY      ANY    /genre/{id}
  app_genre_edit            GET|POST   ANY      ANY    /genre/{id}/edit
  app_genre_delete          POST       ANY      ANY    /genre/{id}
  jeux_afficher             ANY        ANY      ANY    /jeux
  jeux_ajouter              ANY        ANY      ANY    /jeux/ajouter
  jeux_demandermodifier     ANY        ANY      ANY    /jeux/demandermodifier
  jeux_validermodifier      ANY        ANY      ANY    /jeux/validermodifier
  jeux_supprimer            ANY        ANY      ANY    /jeux/supprimer
  app_marque_index          GET        ANY      ANY    /marque/
  app_marque_new            GET|POST   ANY      ANY    /marque/new
  app_marque_show           GET        ANY      ANY    /marque/{id}
  app_marque_edit           GET|POST   ANY      ANY    /marque/{id}/edit
  app_marque_delete         POST       ANY      ANY    /marque/{id}
  app_pegi_index            GET        ANY      ANY    /pegi/
  app_pegi_new              GET|POST   ANY      ANY    /pegi/new
  app_pegi_show             GET        ANY      ANY    /pegi/{id}
  app_pegi_edit             GET|POST   ANY      ANY    /pegi/{id}/edit
  app_pegi_delete           POST       ANY      ANY    /pegi/{id}
  app_plateforme_index      GET        ANY      ANY    /plateforme/
  app_plateforme_new        GET|POST   ANY      ANY    /plateforme/new
  app_plateforme_show       GET        ANY      ANY    /plateforme/{id}
  app_plateforme_edit       GET|POST   ANY      ANY    /plateforme/{id}/edit
  app_plateforme_delete     POST       ANY      ANY    /plateforme/{id}
  app_tournoi_index         GET        ANY      ANY    /tournoi/
  app_tournoi_new           GET|POST   ANY      ANY    /tournoi/new
```

Tester les nouveaux CRUD /tournoi et /cat/tournois.

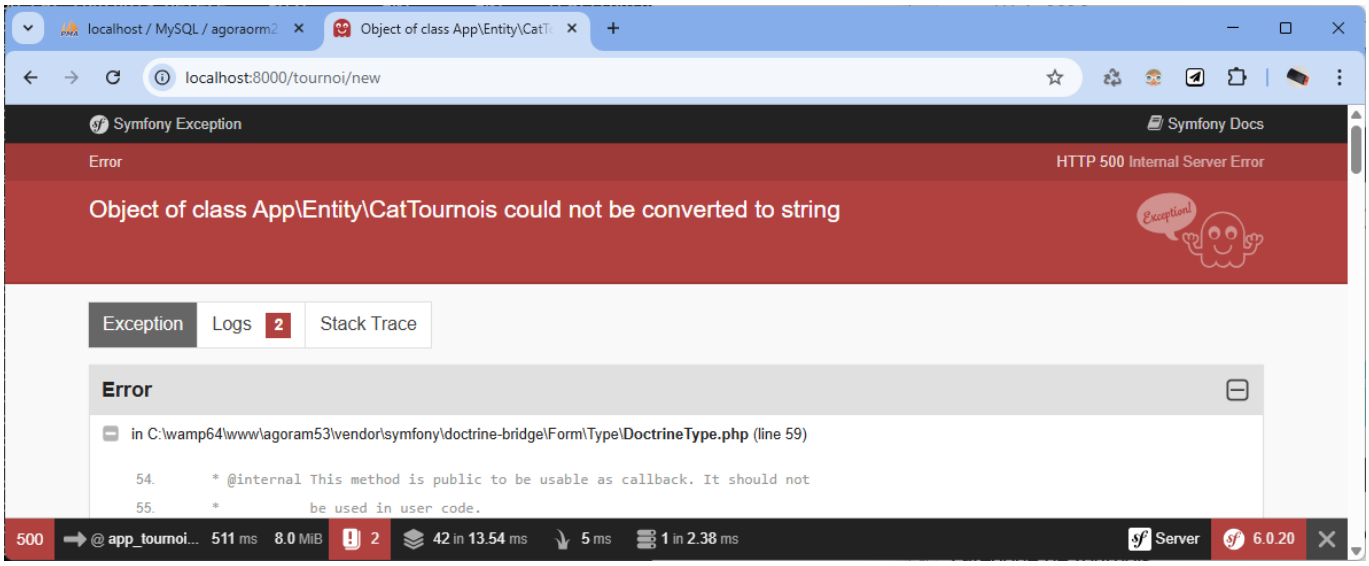
Commencer par créer des catégories de tournois, en saisissant l'URL : http://localhost:8000/cat/tournois



Puis créer des tournois associés à cette nouvelle catégorie de tournois, en saisissant l'URL : http://localhost:8000/tournoi



L'ouverture du formulaire de création des tournois provoque une erreur relative aux types de catégories de tournois :



2.4.4 Convection du type de l'objet :CatTournois

Il faut alors ouvrir l'entité CatTournois et ajouter une méthode qui permet de convertir l'objet en string comme le souhaite le formulaire de création :

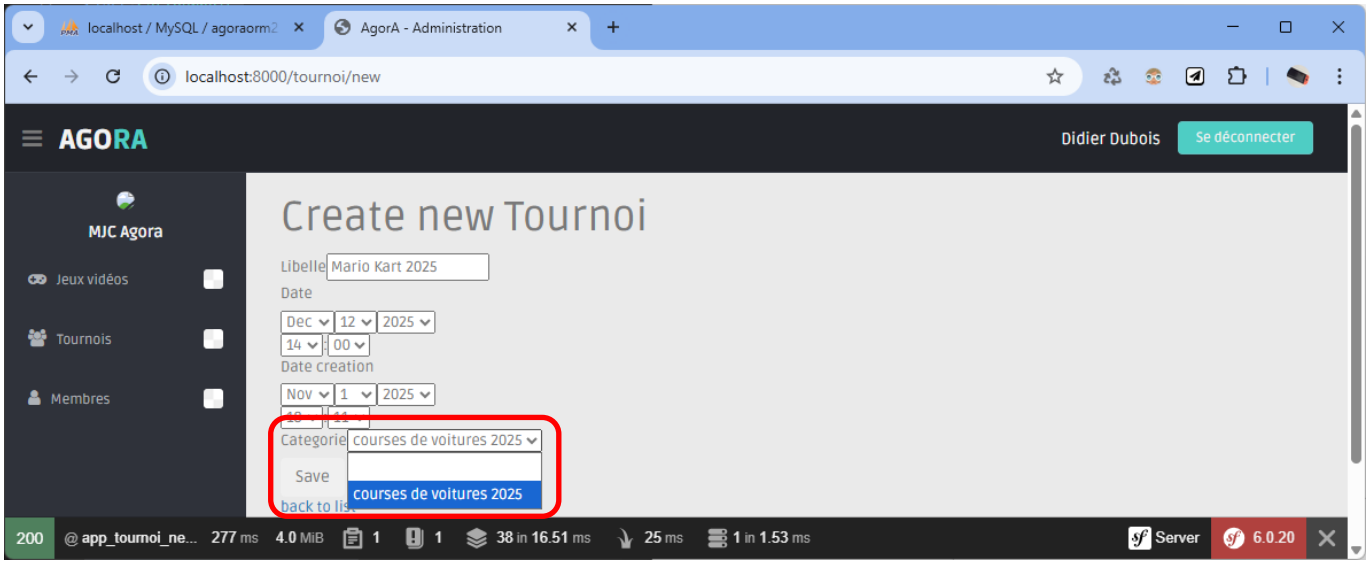
Fichier agora53/src/Entity/CatTournois.php

```php
<?php

namespace App\Entity;

use App\Repository\CatTournoisRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

...

    public function removeTournoi(Tournoi $tournoi): static
    {
        if ($this->tournois->removeElement($tournoi)) {
            // set the owning side to null (unless already changed)
            if ($tournoi->getCategorie() === $this) {
                $tournoi->setCategorie(null);
            }
        }

        return $this;
    }

    public function __toString(): string
    {
        return $this->libelle;
    }
}
```

Lorsque Symfony construit le formulaire, il génère des lignes HTML avec PHP du type :
echo "<option value='1'>" . (string)$catTournois . "</option>" ;

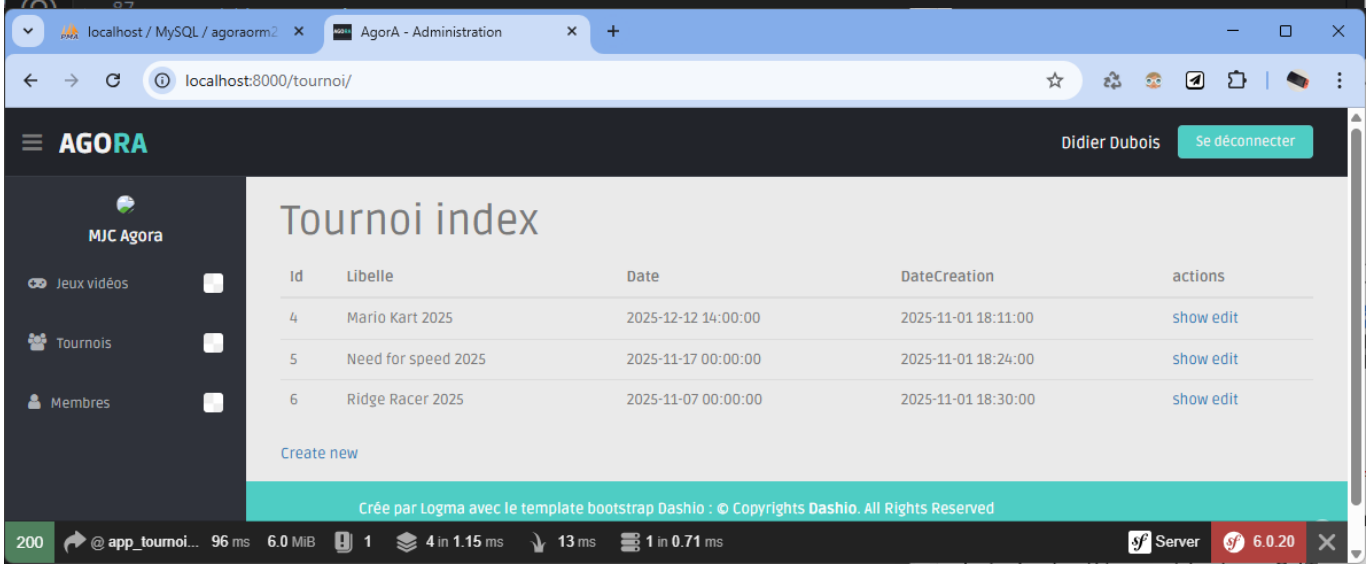
C'est ce cast « string » sur $catTournois qui ne passe pas, car rien n'est prévu pour convertir cet objet en string.
En ajoutant cette méthode public function __toString(): string { return $this->libelle; }, ça renvoie le champ libelle de l'objet CatTournois quand un cast « string » est demandé.

Vous pouvez de nouveau créer des tournois à cette nouvelle catégorie de tournois, en saisissant l'URL : http://localhost:8000/tournoi. La liste des catégories de Tournoi est correctement remplie avec des (string)$cattournois :



Remarquer que les champs qui servent à déterminer la date de création sont positionné sur la date du jour grâce à l'ajout de « $this->dateCreation = new \DateTime('now'); » fait précédemment à la fin de l'entité Tournoi :

```
70          public function __construct()
71          {
72              $this->dateCreation = new \DateTime('now');
73          }
74
75      public function getCategorie(): ?CatTournois
76      {
77          return $this->categorie;
78      }
79
80      public function setCategorie(?CatTournois $categorie): static
81      {
82          $this->categorie = $categorie;
83
84          return $this;
85      }
86  }
```



2.4.5 Ajout des nouveaux formulaires au menu

Il faut désormais ajouter les nouveaux formulaires au menu. Pour cela, il faut éditer templates/menu.html.twig :

```
C: > wamp64 > www > agoram53 > templates > menu.html.twig
8   <div id="sidebar" class="nav-collapse">
10      <ul class="sidebar-menu" id="nav-accordion">
11          <p class="centered"><a href="{{path('accueil')}}"><img src="assets/img/greece-116281
12          <h5 class="centered">MJC Agora</h5>
13          <li class="sub-menu">
14              <a{% if menuActif is defined and menuActif == 'Jeux' %} class="active" {% endif
15                  <i class="fa fa-gamepad"></i>
16                  <span>Jeux vidéos</span>
17              </a>
18              <ul class="sub">
19                  <li><a href="{{path('jeux_afficher')}}">Jeux</a></li>
20                  <li><a href="{{path('app_genre_index')}}">Genres</a></li>
21                  <li><a href="{{path('app_plateforme_index')}}">Plateformes</a></li>
22                  <li><a href="{{path('app_marque_index')}}">Marques</a></li>
23                  <li><a href="{{path('app_pegi_index')}}">Pegi</a></li>
24              </ul>
25          </li>
26          <li class="sub-menu">
27              <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% en
28                  <i class="fa fa-group"></i>
29                  <span>Tournois</span>
30              </a>
31              <ul class="sub">
32                  <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
33                  <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></
34              </ul>
35          </li>
```

Il faut maintenir le menu « Tournois » ouvert lorsque l'on ouvre un de ces 2 formulaires CRUD, pour cela il faut ajouter « 'menuActif' => 'Tournois', » à la fin des appels des vues des contrôleurs TournoisController.php et CatTournoisController.php

```
C: > wamp64 > www > agoram53 > src > Controller > TournoiController.php
1   <?php
2
3   namespace App\Controller;
4
5   use App\Entity\Tournoi;
6   use App\Form\TournoiType;
7   use App\Repository\TournoiRepository;
8   use Doctrine\ORM\EntityManagerInterface;
9   use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
10  use Symfony\Component\HttpFoundation\Request;
11  use Symfony\Component\HttpFoundation\Response;
12  use Symfony\Component\Routing\Annotation\Route;
13
14  #[Route('/tournoi')]
15  class TournoiController extends AbstractController
16  {
17      #[Route('/', name: 'app_tournoi_index', methods: ['GET'])]
18      public function index(TournoiRepository $tournoiRepository): Response
19      {
20          return $this->render('tournoi/index.html.twig', [
21              'tournois' => $tournoiRepository->findAll(), 'menuActif' => 'Tournois',
22          ]);
23      }
```

```php
<?php

namespace App\Controller;

use App\Entity\CatTournois;
use App\Form\CatTournoisType;
use App\Repository\CatTournoisRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/cat/tournois')]
class CatTournoisController extends AbstractController
{
    #[Route('/', name: 'app_cat_tournois_index', methods: ['GET'])]
    public function index(CatTournoisRepository $catTournoisRepository): Response
    {
        return $this->render('cat_tournois/index.html.twig', [
            'cat_tournois' => $catTournoisRepository->findAll(), 'menuActif' => 'Tournois',
        ]);
    }
```

Ceci car le test dans le template menu.html.twig qui maintient le menu Tournois ouvert vérifie si une variable « menuActif » vaut « Tournois » :

```html
<!--sidebar start-->
<aside>
<div id="sidebar" class="nav-collapse">
    <!-- sidebar menu start-->
    <ul class="sidebar-menu" id="nav-accordion">
        <p class="centered"><a href="{{path('accueil')}}"><img src="/assets/img/greece-11628
        <h5 class="centered">MJC Agora</h5>
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Jeux' %} class="active" {% endif
                <i class="fa fa-gamepad"></i>
                <span>Jeux vidéos</span>
            </a>
            <ul class="sub">
                <li><a href="{{path('jeux_afficher')}}">Jeux</a></li>
                <li><a href="{{path('app_genre_index')}}">Genres</a></li>
                <li><a href="{{path('app_plateforme_index')}}">Plateformes</a></li>
                <li><a href="{{path('app_marque_index')}}">Marques</a></li>
                <li><a href="{{path('app_pegi_index')}}">Pegi</a></li>
            </ul>
        </li>
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% en
                <i class="fa fa-group"></i>
                <span>Tournois</span>
            </a>
```

2.4.6 Charter et franciser les nouveaux formulaires

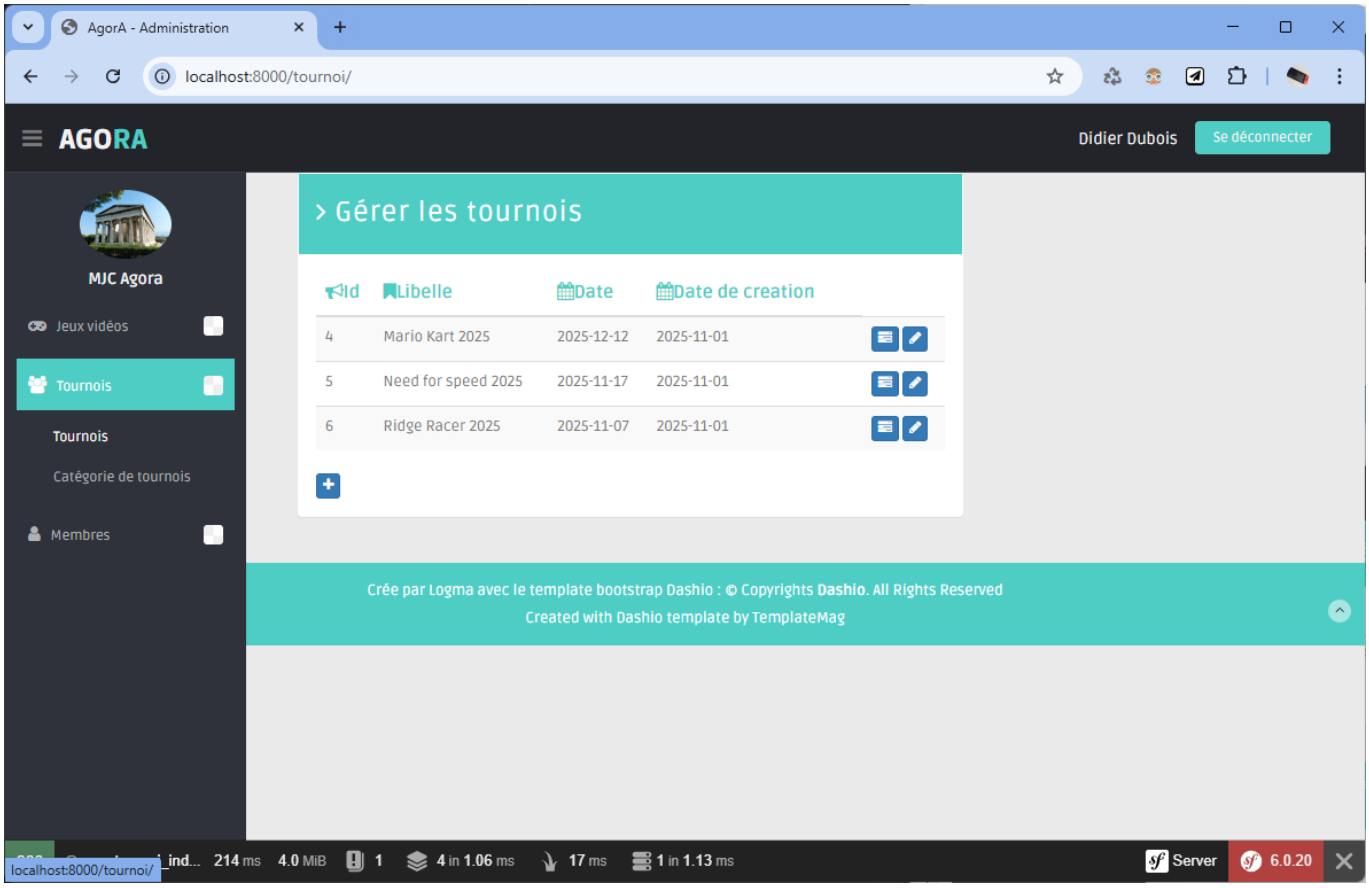
Modifier le fichier templates/tournoi/index.html.twig pour charter et franciser le CRUD pur gérer les tournois :

Fichier agora53/templates/tournoi/index.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Tournoi index{% endblock %}
```

```
{% block body %}
<!-- page start-->
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Gérer les tournois</h3>
        </div>
        <div class="panel-body">
    <table class="table table-striped table-advance table-hover">
        <thead>
            <tr class="tableau-entete">
                <th><i class="fa fa-bullhorn"></i>Id</th>
                <th><i class="fa fa-bookmark"></i>Libelle</th>
                <th><i class="fa fa fa-calendar"></i>Date</th>
                <th><i class="fa fa fa-calendar"></i>Date de création</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
        {% for tournoi in tournois %}
            <tr>
                <td>{{ tournoi.id }}</td>
                <td>{{ tournoi.libelle }}</td>
                <td>{{ tournoi.date ? tournoi.date|date('Y-m-d H:i:s') : '' }}</td>
                <td>{{ tournoi.dateCreation ? tournoi.dateCreation|date('Y-m-d H:i:s') : '' }}</td>
                <td>
                    <a href="{{ path('app_tournoi_show', {'id': tournoi.id}) }}"><button class="btn btn-
primary btn-xs"><i class="fa fa-tasks"></i></button></a>
                    <a href="{{ path('app_tournoi_edit', {'id': tournoi.id}) }}"><button class="btn btn-primary
btn-xs"><i class="fa fa-pencil"></i></button></a>
                </td>
            </tr>
        {% else %}
            <tr>
                <td colspan="5">aucun tournoi enregistré en base de données</td>
            </tr>
        {% endfor %}
        </tbody>
    </table>

    <a href="{{ path('app_tournoi_new') }}"><button class="btn btn-primary btn-
xs"><i class="fa fa-plus"></i></button></a>
            </div><!-- fin div panel-body-->
    </section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->
{% endblock %}
```

Suivre la méthodologie mise en œuvre pour charter et franciser le CRUD de gestion des genres pour améliorer les écrans de création, modification et liste.



Modifier le fichier templates/tournoi/index.html.twig pour charter et franciser le CRUD pur gérer les tournois :

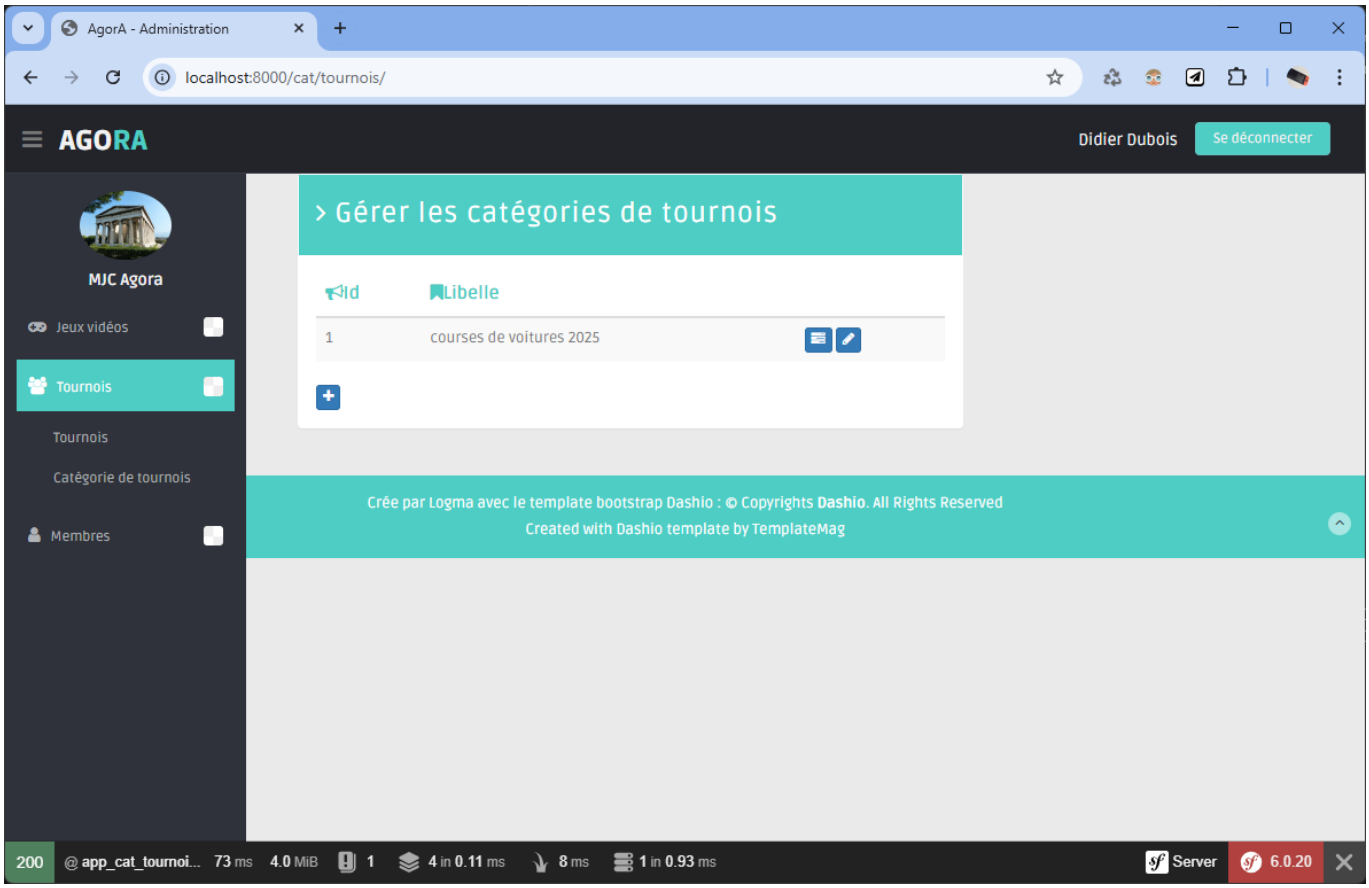
Fichier agora53/templates/cat_tournois/index.html.twig

```
{% extends 'base.html.twig' %}

{% block body %}
<!-- page start-->
<div class="col-sm-8">
    <section class="panel">
        <div class="chat-room-head">
            <h3><i class="fa fa-angle-right"></i> Gérer les catégories de tournois</h3>
        </div>
        <div class="panel-body">
    <table class="table table-striped table-advance table-hover">
        <thead>
            <tr class="tableau-entete">
                <th><i class="fa fa-bullhorn"></i>Id</th>
                <th><i class="fa fa-bookmark"></i>Libelle</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
        {% for cat_tournoi in cat_tournois %}
            <tr>
                <td>{{ cat_tournoi.id }}</td>
                <td>{{ cat_tournoi.libelle }}</td>
                <td>
```

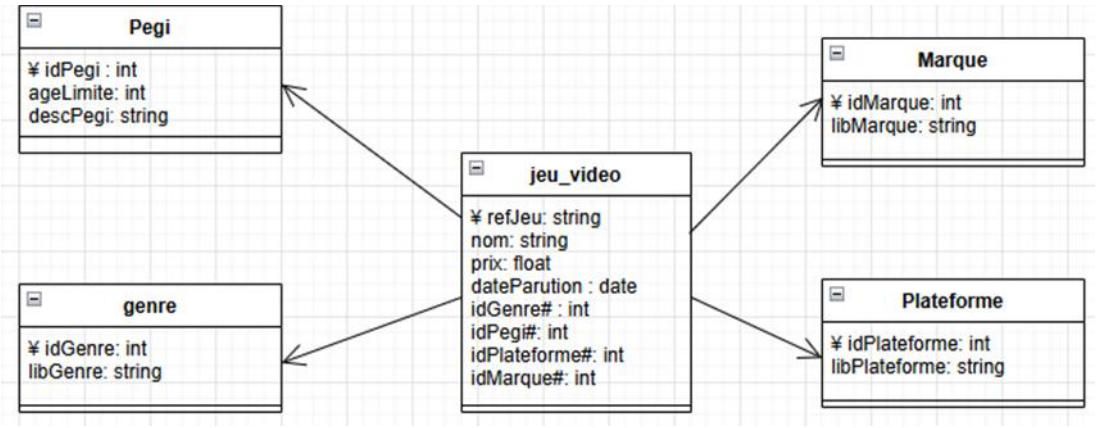
```
                    <a href="{{ path('app_cat_tournois_show', {'id':
cat_tournoi.id}) }}"><button class="btn btn-primary btn-xs"><i class="fa fa-
tasks"></i></button></a>
                    <a href="{{ path('app_cat_tournois_edit', {'id':
cat_tournoi.id}) }}"><button class="btn btn-primary btn-xs"><i class="fa fa-
pencil"></i></button></a>
                </td>
            </tr>
        {% else %}
            <tr>
                <td colspan="3">aucune catégorie de tournois enregistrée en base de données</td>
            </tr>
        {% endfor %}
        </tbody>
    </table>
    <a href="{{ path('app_cat_tournois_new') }}"><button class="btn btn-primary
btn-xs"><i class="fa fa-plus"></i></button></a>
            </div><!-- fin div panel-body-->
    </section><!-- fin section tournois-->
</div><!--fin div col-sm-8-->
{% endblock %}
```

Suivre la méthodologie mise en œuvre pour charter et franciser le CRUD de gestion des genres pour améliorer les écrans de création, modification et liste.

Tester :



3. A vous de jouer

Vous avez désormais une méthodologie pour mettre en œuvre une association many to one Doctrine entre deux entités.
Appliquez maintenant cette démarche sur le CRUD jeux qui est également en association many to one avec :

- genres
- plateformes
- PEGIs
- et marques